Exercise - Pandas Sql

Start Assignment

- Due Sunday by 11:59pm
- Points 20
- Submitting a file upload

Pandas CSV -> SQLite Viewer (GUI Required)

Overview

In the previous SQLite assignment, you loaded CSV data using Python, converted the data into dictionaries, generated SQL, and used SQL commands to build and work with database tables.

This assignment introduces **pandas**. Pandas provides higher-level tools for loading and working with structured data. Instead of converting CSV rows into dictionaries and manually building the SQLite table, you will use a pandas **DataFrame**.

The new pipeline is:

```
CSV file
    ->
pandas DataFrame
    ->
SQLite table
    ->
SQL SELECT query
    ->
pandas DataFrame
    ->
Tkinter display
```

The purpose of the assignment is to connect pandas to tools you have already used: CSV files, SQLite, SQL, and Tkinter.

The pandas operations are only one part of the assignment. Your program should demonstrate good object-oriented design. The work should be encapsulated in classes with clear responsibilities rather than written as one procedural script.

Your design will be evaluated for **modularity, robustness, scalability, correctness, clarity, and style**. A program that produces the correct output but does not demonstrate these design principles does not fully meet the requirements of the assignment.

---

1. Load a CSV File with Pandas

Use `pandas.read_csv()` to load a CSV file into a DataFrame.

```
df = pd.read_csv("Presidents.csv")
```

The DataFrame now represents the rows and columns from the CSV file. You no longer need to use the Python `csv` module or convert the rows into a list of dictionaries.

Your program should be able to provide basic information about the DataFrame, including:

- column names
- number of rows
- data types

The pandas operations should be handled by an appropriate class rather than being scattered throughout the GUI or `main`.

---

2. Convert the DataFrame to SQLite

Create an SQLite database connection using the `sqlite3` module.

Use the pandas `to_sql()` method to write the DataFrame into an SQLite table.

```
df.to_sql(
    "Presidents",
    conn,
    if_exists="replace",
    index=False
)
```

Pandas now handles the work that you previously performed with CREATE TABLE and INSERT statements.

The SQLite table should contain the same columns and rows as the DataFrame.

Database responsibilities should be encapsulated rather than mixed directly into GUI event handlers.

---

3. Query the SQLite Database

Once the DataFrame has been stored in SQLite, use a SQL `SELECT` statement to retrieve data from the table.

For example:

```
SELECT *
FROM Presidents;
```

Your program should also demonstrate at least one SELECT query that retrieves only part of the table using a condition.

For example:

```
SELECT President, State
FROM Presidents
WHERE State = 'Ohio';
```

The particular query will depend on the CSV file being used.

---

4. Load SQL Results into a DataFrame

Use pandas to execute the SELECT query and return the results as another DataFrame.

```
results = pd.read_sql_query(sql, conn)
```

Notice that pandas appears at both ends of the database operation:

```
DataFrame -> SQLite -> SQL SELECT -> DataFrame
```

The original DataFrame comes from the CSV file. The second DataFrame contains the rows returned by the SQL query.

---

5. Tkinter GUI

Create a Tkinter interface that acts as the driver for the program. The GUI coordinates the program but should not contain the pandas and database implementation itself.

The GUI should allow the user to:

- load a CSV file
- convert the CSV into a pandas DataFrame
- store the DataFrame in an SQLite table
- execute a SELECT query
- display the query results

Minimum GUI Elements

- Load CSV button
- Create SQLite Table button
- Run Query button
- SQL query display
- query results display

Use a `ttk.Treeview` or another appropriate Tkinter widget to display the rows and columns returned by the query.

---

6. DataFrame-Based Display

The query results are returned as a DataFrame. Use the DataFrame structure to build the GUI display dynamically.

Do not hard-code the column names into the Treeview. Obtain them from the DataFrame.

For example, the program can obtain the columns from:

```
results.columns
```

The program should also obtain the rows from the DataFrame rather than assuming a particular number or arrangement of columns.

This allows the same GUI to display different CSV files and different SELECT queries.

---

7. Validation and Robustness

Your program should handle expected problems cleanly rather than failing with an unhandled exception.

Your program should confirm that:

- the CSV file loads successfully into a DataFrame
- the DataFrame contains the expected columns and rows
- the DataFrame is successfully written to SQLite
- the SQLite table contains the expected data
- the SELECT query returns a DataFrame
- the query results are displayed correctly in Tkinter

Consider situations such as a missing or unreadable file, an invalid query, an unavailable database, or an operation attempted before the required data has been loaded. Errors should be reported clearly to the user.

---

8. CSV Files

Your program should work with the supplied CSV files without hard-coding their columns:

- CO2.csv
- Dwarfs.csv
- Elements.csv
- Presidents.csv
- Temperature.csv

The table name may be derived from the CSV filename. For example, `Presidents.csv` could create an SQLite table named `Presidents`.

Your design should not depend on one particular CSV file. Loading another supplied file should not require rewriting the program.

---

9. Object-Oriented Program Design

This assignment is expected to use a modular, class-based design. Classes should have clear responsibilities and should encapsulate the data and operations that belong together.

Appropriate responsibilities include:

- loading and managing CSV and DataFrame data
- writing DataFrames to SQLite
- executing SQL queries and returning results
- displaying DataFrame results
- managing the Tkinter interface

You may decide how these responsibilities should be divided among your classes. The goal is not a particular number of classes. The goal is a design in which each class has a clear purpose.

Design Expectations

- **Modularity:** Separate responsibilities into appropriate classes and methods.
- **Robustness:** Validate operations and handle expected errors without crashing the program.
- **Scalability:** The design should work with different CSV files, columns, and queries without requiring major changes.
- **Correctness:** Data should move correctly through the complete CSV -> DataFrame -> SQLite -> query -> display pipeline.
- **Clarity:** Class, method, and variable names should clearly describe their purpose. Program flow should be easy to follow.
- **Style:** Avoid duplicated code, unnecessarily long methods, excessive global data, and large blocks of procedural code.

Keep main Small

The main part of the program should create the necessary objects and start the application. It should not contain the implementation of the CSV, pandas, SQLite, SQL, or display operations.

In other words, avoid writing the assignment as a large script that happens to use pandas. The organization of the program is part of the assignment.

---

10. Notebook Development

Use the Jupyter notebooks from this module as the basis for assembling your program. The notebook examples demonstrate the individual pandas, SQLite, SQL, and Tkinter techniques needed for the assignment.

Build and test the pieces separately before combining them into the complete application. Comment your code where the purpose or reasoning may not be obvious.
